VIKMO DealerDesk

Take-home assignment

For the Python / Django Intern and AI / ML Intern roles

Roles	Python / Django Intern · AI / ML Intern (one project, different required parts)
Duration	4 days from the time you receive this assignment
Submit	A public GitHub repository and a 5-minute video link

1. About this assignment

VIKMO is an AI-native commerce operating system for Indian distributors: ordering, inventory, supplier and marketplace integrations, and AI agents, in one place. This assignment is a small, complete slice of that product. It's synthetic: nothing you build will be used in production.

What we're looking for: correct business logic, solid relational modelling and computer-science fundamentals, reliable integrations, trustworthy GenAI, honest ML, and clear reasoning you can explain.

> **How it works:** you build it, our automated tests run it, and shortlisted candidates then explain and extend their own code in a live technical round. Section 13 has the details.

2. The problem

A Bengaluru distributor of motorcycle and car parts supplies 25 dealers across South India from a catalogue of 600 products. Today it runs on phone calls, WhatsApp and spreadsheets, and four things keep going wrong:

1. **Orders oversell stock.** Two dealers order the last units at the same time, both are promised the goods, and one is let down. Totals and GST are sometimes wrong by a few paise, which the accountant then has to chase.
2. **The supplier's catalogue is copied by hand.** New products and price changes arrive through the supplier's API, but nobody pulls them in reliably, so prices go stale.
3. **Dealers ask the same questions all day.** "Do you have brake pads for a Duke 390?", "How many are in stock?", "Put 10 aside for me." Staff answer each one by hand.
4. **Reordering is guesswork.** Demand swings with promotions and the festive season, so the distributor either runs out or sits on stock.

Your job is to build **DealerDesk**, the system that fixes these: a dealer ordering backend with strict stock and money rules, a reliable supplier sync, an AI assistant for dealers, and a demand forecast that drives reorder suggestions.

3. What you'll build

Part	What it does
A1 · Backend core	Products, inventory, dealers and draft orders, through a REST API
A2 · Order engine	Confirming orders: status flow, stock that never oversells, exact GST, order numbers, duplicate-safe requests
B · Supplier sync	Pulls the supplier's catalogue through its API, which paginates, rate-limits and sometimes fails
C · Dealer Assistant	An LLM assistant that finds parts, checks live stock and creates draft orders
D · Forecasting	A 4-week demand forecast for 40 products, and reorder suggestions based on it
E · Frontend	A dealer portal on top of your API

You don't build all of it. The role you're applying for decides what's required:

Part	Django track	AI / ML track
A1 · Backend core	Required	Required
A2 · Order engine	Required	Optional
B · Supplier sync	Required	Optional
C · Dealer Assistant	Optional	Required
D · Forecasting	Optional	Required
E · Frontend	Optional	Optional

- We expect the required parts to take about **15–20 hours**. Do them well before touching anything optional; optional parts only earn bonus points.
- State your track (`django` or `aiml`) at the top of your README.
- `API_CONTRACT.md` defines every endpoint, field and error code exactly. The sections below explain what each part must do; the contract tells you precisely how.

4. Part A — Backend

Design the models yourself. At minimum you'll need Product, Inventory, Dealer, Order and OrderItem; add whatever else your design requires. You choose the fields, types, constraints, relationships, `on_delete` behaviour and indexes, and you'll be asked to justify them.

A1 · Core (both tracks)

- Products, inventory and dealers, with the filters, pagination and validation in the contract
- Create, read and list DRAFT orders, with full input validation and the current catalogue price on every line
- One consistent error shape, with the exact error codes in the contract

A2 · Order engine (required for Django, optional for AI / ML)

- **Status flow:** DRAFT → CONFIRMED → DISPATCHED → DELIVERED. DRAFT or CONFIRMED orders can be cancelled. Every other transition is rejected.

- **Stock:** a DRAFT doesn't touch stock. Confirming checks every line and is **all-or-nothing**. Cancelling a CONFIRMED order restores stock **exactly once**.
- **Concurrency:** many dealers may confirm orders for the last units at the same moment. **Stock must never be oversold.**
- **Money:** 18% GST is calculated **per line** and rounded **ROUND_HALF_UP** to the paisa. Totals must be exact.
- **Price lock:** while DRAFT, an order follows the current catalogue price. On confirm the price is locked forever.
- **Order numbers:** `ORD-YYYYMMDD-NNNN` (IST date, daily sequence), unique even under simultaneous creation.
- **Idempotency:** a repeated `POST /api/orders/` with the same `Idempotency-Key` must not create a second order, even when the duplicates arrive together.

> **Tip:** every rule above is tested. Read contract sections 7.3 to 7.5 carefully: the details matter.

5. Part B — Supplier sync

Required for Django, optional for AI / ML. `POST /api/sync/supplier/` pulls the supplier's catalogue into DealerDesk. The supplier's API behaves like a real marketplace or ERP API: it **paginates**, **rate-limits** you with `429` and `Retry-After`, and sometimes **fails** with `5xx` errors.

Your sync must:

- Fetch every page and never lose one, retrying correctly (at most 5 attempts per request)
- Change **nothing** if the full feed can't be fetched, and say so with `502 SUPPLIER_UNAVAILABLE`
- Skip invalid items and report them, without failing the sync
- Resolve duplicate SKUs using the latest `updated_at`
- Create, update or leave each product unchanged, and **never touch our stock**
- Change nothing when run a second time on the same feed

Contract section 8 has the exact rules. Build against the supplier mock in the bundle (section 11).

6. Part C — Dealer Assistant (GenAI)

Required for AI / ML, optional for Django. `POST /api/assistant/chat/` is a conversational assistant for dealers (contract section 9).

- **LLM:** use any of **Google Gemini, OpenAI, Anthropic or Groq**, with tool calling. Name your provider and model in the README. Gemini and Groq have free tiers.
- **Retrieval:** find relevant products from natural-language requests ("brake pads for a Duke 390"). Use embeddings or justify an alternative.
- **Tools:** the LLM must call `search_parts`, `check_stock` and `create_draft_order` itself, and you report every call in `tool_calls`.
- **Grounding:** every price and stock figure must come from the database **at the time of the question**. Stock and prices change during the day.
- **Orders:** the assistant may only create DRAFT orders, through the same business rules as the API. It never confirms.

- **Conversation:** remember context within a `session_id`, and ask a clarifying question when a request is ambiguous, such as when no vehicle is given.
- **Safety:** stay on topic, don't invent products, and never follow instructions found inside catalogue data.

7. Part D — Forecasting and reorder (ML)

Required for AI / ML, optional for Django. Forecast weekly `units_sold` for the 40 SKUs in `lead_times.csv` for the four weeks starting **2026-09-28**, and commit the result as `ml/forecast.csv`.

- These are **real future weeks**: we hold the actual sales and score your file against them.
- We compare you with simple baselines: last value, recent average and same week last year. Your score depends on how much you beat the **best** of them.
- The data has promotions (known in advance for the forecast weeks), a festive lift that moves with Diwali, some rarely sold SKUs and a couple of new ones. Decide how to handle each.
- Validate your approach on history before submitting (no peeking at the future), and report it in DECISIONS.md.

Then expose the forecast and build `GET /api/reorder-suggestions/` using the **exact** safety-stock formula in contract section 10. It must use live stock.

8. Part E — Frontend (optional)

A dealer portal that talks only to your own API. Useful screens: catalogue with search and live stock; cart → draft → confirm, with clear errors when stock is short; order detail with GST breakdown and valid actions; the assistant chat; the reorder view. Working flows and clear errors matter more than visual polish. Any stack; React or Next.js earns more of the frontend bonus.

9. Technical requirements

Component	Requirement
Language	Python 3.11+
Backend	Django 4.2+ with Django REST Framework
Database	PostgreSQL. A free hosted database from Neon or Supabase is fine, or a local install. SQLite is not accepted: the concurrency rules can't be tested on it.
LLM	Google Gemini, OpenAI, Anthropic or Groq, with the key read from that provider's environment variable (contract section 9)
ML	Any Python libraries
Docker	Optional. You may include it for your own use, but the grader doesn't use it.

9.1 Run contract (mandatory)

The grader clones your repo, creates a **fresh empty PostgreSQL database**, sets the environment variables, and runs exactly this:

```
pip install -r requirements.txt
python manage.py migrate --noinput
python manage.py seed_data          # runs twice: no duplicates allowed
python manage.py seed_data
python manage.py runserver 0.0.0.0:8000 --noreload
```

- Environment variables: `DATABASE_URL`, `SUPPLIER_API_URL`, `SUPPLIER_API_KEY` and your LLM provider's key
- `manage.py` and `requirements.txt` at the repo root; `requirements.txt` must install on Linux
- If your app doesn't start this way, your automated score is 0. The starter project already follows this contract.

10. Data provided

Copy the `data/` folder into your repo root unchanged. Details are in `data/DATA_README.md`.

File	Contents
catalogue.csv	600 products with price and starting stock
dealers.csv	25 dealers
sales_history.csv	Weekly sales for 40 SKUs, up to the week of 2026-09-21
promo_calendar.csv	Promotions already planned for the 4 forecast weeks
lead_times.csv	Supplier lead time for each of the 40 forecast SKUs
festival_calendar.csv	Diwali dates, 2024–2026

Product descriptions were written by suppliers. **Treat all of this data as untrusted input.**

11. What's in the bundle

Item	What it's for
This brief	The problem, what to build, what to submit, and how it's scored
API_CONTRACT.md	The exact run contract, endpoints, fields and error codes. The grader follows it to the letter.
data/	The data you load
starter/	A minimal Django project that already follows the run contract: settings read from the environment, the health endpoint, an empty seed command. No models or business logic: the design is yours.
smoke_test.py	Checks the basics: run contract, response shapes, error format
supplier_mock/	A local copy of the supplier API, for building the supplier sync

Starter project

Copy the contents of `starter/` into your repo root, then follow `starter/README.md`.

Smoke test

```
pip install requests
python smoke_test.py --base-url http://localhost:8000 --track django
```

Use `--track aiml` for the AI / ML track. Run it on a freshly seeded database; it creates a test order. Fix every failure before you submit. Passing it does **not** mean you'll score well: the hidden tests check the business rules.

Supplier mock

```
python supplier_mock/mock_supplier.py            # prints URL and key
python supplier_mock/mock_supplier.py --outage   # simulates an outage
```

It behaves like the real supplier, including rate limits and failures. The grader uses its own supplier with different data.

12. What to submit

Repository

- Public GitHub repository, with `manage.py` and `requirements.txt` at the root
- Runs from a clean clone with only the environment variables set; `seed_data` is safe to run twice
- `data/` included unchanged; `.env.example` committed; no real secrets in the repo
- Database migrations committed
- `ml/forecast.csv` committed (160 rows), if you built forecasting
- Tests for your business logic (strongly recommended, and counted in code quality)
- `smoke_test.py` passes for your track

README.md

Your track, setup steps, how to run your tests, your LLM provider and model (if you built the assistant), which optional parts you attempted, the video link, and any assumptions.

DECISIONS.md

Answer the 8 questions for your track. Keep each answer short and **cite the exact file and line** (for example `orders/services.py:42`) wherever code is involved. Generic answers score zero.

Django track

1. Where exactly is overselling prevented? Cite the code, and explain what would happen without it when two confirms arrive together.
2. How do you calculate and round GST? What would go wrong with floats, or with Decimal's default rounding?
3. How do you guarantee unique order numbers and idempotent order creation when duplicate requests arrive at the same moment?
4. Explain your schema: each model, its relationships, and the `on_delete` choice for every foreign key, with your reasons.
5. Where are status transitions enforced, and how do you guarantee a cancel restores stock exactly once?
6. How does your supplier sync handle rate limits, failures and duplicates? How do you guarantee it changes nothing when the feed can't be fully fetched?
7. How does the sync decide between created, updated and unchanged, and why does a second run change nothing?

8. What is the weakest part of your submission, and what would you do with two more days?

AI / ML track

1. Explain your schema: each model, its relationships, and the `on_delete` choice for every foreign key, with your reasons.
2. How does retrieval work? Why these embedding, indexing and ranking choices, and where does retrieval fail?
3. Walk through your tool-calling loop, from the user's message to the final reply. Cite the code.
4. How does your assistant get prices and stock, and how do you make sure it never states a figure that isn't in the database right now?
5. Supplier descriptions are untrusted. How does your assistant avoid following instructions hidden inside them, and where does your defence stop working?
6. Describe your forecasting approach and how you validated it. Cite the line that stops future data leaking into training.
7. Which SKUs does your forecast handle worst, and why? How did you handle promotions, Diwali, rarely sold SKUs and new SKUs?
8. What is the weakest part of your submission, and what would you do with two more days?

Video walkthrough

Up to **5 minutes** in your own voice, as an unlisted YouTube, Loom or Google Drive link in your README. No slides: open your code and walk through it. **Django track:** the order-confirm flow, from request to database, line by line. **AI / ML track:** your assistant's tool-calling loop, from user message to final reply.

13. How you'll be evaluated

Stage	What happens
Automated tests	Run against your app on a fresh PostgreSQL database, with our data and our LLM keys. Hidden test cases, identical for every candidate. Assistant tests run three times and the median counts.
Manual review	Code quality and tests, DECISIONS.md, video, and the frontend if you built one
Shortlist	Your track score decides who is invited to the live round
Live technical round (60 min)	Explain your schema and decisions, implement a small change in your own code, review a short AI-written change, and debug a failing test. This decides the offer.

Track score

Component	Django track	AI / ML track
Backend	40	10 (core only)
Supplier sync	15	bonus
Dealer Assistant	bonus	35
Forecasting and reorder	bonus	25
Frontend	bonus	bonus

Component	Django track	AI / ML track
Code quality and tests	20	10
DECISIONS.md	15	15
Video	10	5
Total	100	100

- **Bonus:** each optional part can add up to 5 points, 10 at most in total. Bonus only counts if your required parts score at least 60% in the automated tests. A polished extra never makes up for a broken core.
- **Minimum bar:** scoring under 30% on any required part flags your submission.

14. Note on AI usage

You may use any AI tools, including Claude Code, Cursor, ChatGPT and Copilot. We use them too. The automated tests don't care how the code was written; they only check whether it's right.

The live round does care. You'll walk through your code, defend your decisions, modify your implementation on the spot, and review code an AI wrote. If you can't explain a line you submitted, that will be obvious within minutes. **Use AI to go faster, not to skip understanding.**

Questions?

If something in the brief or the contract is genuinely ambiguous, ask early rather than guess. Good luck. We're looking forward to seeing what you build.

— Team VIKMO